CSF Short Notes

Computer Systems Fundamentals (4COSC009C) · IP addressing, partition sector zero and logic gates · typed up from handwritten notes and class photos

1. IP Addressing

An IP calculation asks for three things: (1) the **network address**, (2) the **broadcast address**, and (3) the **valid host range** (from / to).

CIDR is the number after the slash: the count of 1-bits in the subnet mask. e.g. 192.168.1.0/**24**, 10.1.1.32/**28**.

Method

1. Write the IP and the mask in binary.
2. Network address: keep the network bits, set all host bits to 0 (= IP AND mask).
3. Broadcast address: set all host bits to 1.
4. First host = network address + 1; last host = broadcast address - 1.

Example 1 - 10.1.1.32/28

```
10.1.1.32  = 00001010.00000001.00000001.0010|0000   (/28: last 4 bits are host bits)
Network    = 0010 0000 -> 10.1.1.32
Broadcast  = 0010 1111 -> 10.1.1.47
First host = 0010 0001 -> 10.1.1.33
Last host  = 0010 1110 -> 10.1.1.46
```

Example 2 - 192.45.63.156/25

```
Mask /25   = 11111111.11111111.11111111.10000000 = 255.255.255.128
156        = 1|0011100
Network    = 192.45.63.128   (host bits 0000000)
Broadcast  = 192.45.63.255   (host bits 1111111)
Host range = 192.45.63.129 to 192.45.63.254
```

Example 3 - first valid host for 163.56.170.94/22

```
163.56.170.94 = 10100011.00111000.101010|10.01011110
Network       = 10100011.00111000.101010|00.00000000 = 163.56.168.0
First host    = 163.56.168.1
```

IP address classes (first octet)

Class	First octet	Starting address	Ending address
A	0 - 127	0.0.0.0	127.255.255.255
B	128 - 191	128.0.0.0	191.255.255.255
C	192 - 223	192.0.0.0	223.255.255.255
D	224 - 239	224.0.0.0	239.255.255.255
E	240 - 255	240.0.0.0	255.255.255.255

2. Partition Sector Zero (FAT-16 Boot Sector)

Values in the boot sector are stored **little-endian**: reverse the byte order to read them (e.g. `00 02` → `0200` hex = 512).

Boot sector fields (annotated example from class)

```
EB 3C 90                  not used
4D 53 44 4F 53 35 2E 30   "MSDOS5.0" - not used
00 02  -> 02 00           bytes per sector = 512
10                        sectors per cluster = 16
01 00  -> 00 01           size of Reserved Area in sectors = 1
02                        number of FAT tables = 2
00 02  -> 02 00           number of files in the root directory = 512
00 00                     0 -> look at the last entry for the size of the partition
F8                        fixed disk
F1 00  -> 00 F1           size of a FAT table = 241 sectors
3F 00  -> 00 3F           sectors per track = 63
FF 00  -> 00 FF           255 heads - not used
78 02 00 00 -> 00 00 02 78   number of sectors before the start of the partition = 632
88 05 0F 00 -> 00 0F 05 88   size of the partition in sectors = 984456
```

Root directory calculation (worked example from the practice test)

```
1. Partition starts at sector number 1048576
2. Bytes 14-15: 02 00 -> 0002 = 2       Reserved Area (RA)
3. Byte 16:     02       = 2            number of FATs
4. Bytes 22-23: 07 00 -> 0007 = 7       size of each FAT
5. Bytes 28-31: 00 08 00 00 -> 0000 0800 = 8 x 16^2 = 2048   sectors before the partitic
6. RA = 2
7. Size of all FATs = size of each FAT x 2 = 7 x 2 = 14
8. Root directory = 2048 + 2 + 14 = 2064
```

Formulas

- First FAT start sector = sectors before start of partition + RA
- Second FAT start sector = first FAT + size of each FAT
- Root Directory start sector = first FAT + (size of each FAT x 2)

3. Logic Gates

- Logic gates are the basic building blocks of any digital system.
- They are implemented inside integrated circuits (ICs).
- Each gate has a logical operation, shown with a circuit diagram and a truth table.
- 0 = low voltage, 1 = high voltage.

Basic gates

AND and OR can have any number of inputs; NOT has exactly one input (unary operator).

A	B	NOT A	AND (A.B)	OR (A+B)
0	0	1	0	0
0	1	1	0	1
1	0	0	0	1
1	1	0	1	1

NOT (logical inverter): Y = $\bar{A}$ (also written A'). AND (logical conjunction): Y = A.B. OR (logical inclusive): Y = A + B.

Derived gates

NAND and NOR can have any number of inputs; XOR and XNOR take 2 inputs.

A	B	NAND	NOR	XOR	XNOR

0	0	1	1	0	1
0	1	1	0	1	0
1	0	1	0	1	0
1	1	0	0	0	1

- **NAND** = AND followed by NOT: Y = (A.B)' (not $\overline{A.B}$).
- **NOR** = OR followed by NOT: Y = (A+B)'. *(The handwritten notes said (A.B)'; that is NAND, so it is corrected here.)*
- **XOR** (A $\oplus$ B): same inputs give 0, different inputs give 1.
- **XNOR** (exclusive NOR): same inputs give 1, different inputs give 0.

Multi-input rules of thumb

- 4-input AND: all four inputs must be 1 to get a high output (1).
- 3-input OR: at least one of the three inputs must be 1 to get output 1.
- 8-input AND: all 8 inputs must be 1 to get 1.